

MVP — DATA ENGINEERING

Relação entre Desenvolvimento Socioeconômico e Pegada Ecológica dos Países

**Construção de um Pipeline de Dados na Nuvem
Arquitetura Medalhão no Databricks**

Aluno: Thompson Fiorio
Pós-graduação em Ciência de Dados
Sprint: Engenharia de Dados
2026

Sumário

1.	Objetivo.....	3
1.1	Perguntas de análise	3
2.	Contexto	3
3.	Arquitetura Bronze/Silver/Gold.....	4
4.	Coleta e carregamento dos dados	5
5.	Modelagem de dados	7
6.	Catálogo de dados	9
7.	Pipeline de dados	11
8.	Qualidade dos dados.....	12
9.	Análises	14
9.1	Correlações entre desenvolvimento e pegadas	14
9.2	Análise por região	15
9.3	Análise por faixa de IDH	16
9.4	Análise por faixa de PIB Per Capita	17
9.5	Países com maiores valores	18
9.6	Outliers	21
9.7	Visualizações recomendadas no Databricks	22
10.	Interpretação dos resultados.....	23
11.	Conclusões.....	25
12.	Limitações e pontos de melhoria	25
13.	Autoavaliação	25

1. Objetivo

O objetivo deste MVP é construir um pipeline de dados em nuvem capaz de coletar, armazenar, tratar, modelar e analisar informações socioeconômicas e ambientais de diferentes países. O estudo busca identificar a relação entre o Índice de Desenvolvimento Humano (IDH) e o PIB Per Capita (GDP Per Capita) com quatro componentes da pegada ecológica: Pegada de Carbono (*Carbon Footprint*), Pegada de Peixe (*Fish Footprint*), Área de Cultivo (*Cropland Footprint*) e Área de Pastagem (*Grazing Footprint*).

Além da análise do problema, o trabalho foi desenvolvido com foco em Engenharia de Dados. Assim, os dados são persistidos no Databricks em tabelas Delta organizadas segundo a arquitetura Medalhão. Python e PySpark são utilizados principalmente nas etapas de ingestão, transformação e qualidade, enquanto a maior parte das análises finais é executada com Spark SQL diretamente sobre tabelas armazenadas no ambiente.

1.1 Perguntas de análise

1. Qual é a relação entre IDH e Pegada de Carbono?
2. Qual é a relação entre PIB Per Capita e Pegada de Carbono?
3. Como IDH e PIB Per Capita se relacionam com Pegada de Peixe?
4. Como IDH e PIB Per Capita se relacionam com Área de Cultivo?
5. Como IDH e PIB Per Capita se relacionam com Área de Pastagem?
6. Como os indicadores se comportam quando os países são agrupados por região?
7. Quais países apresentam valores extremos em relação às variáveis estudadas?
8. Quais relações apresentam maior intensidade segundo o coeficiente de correlação de Pearson?

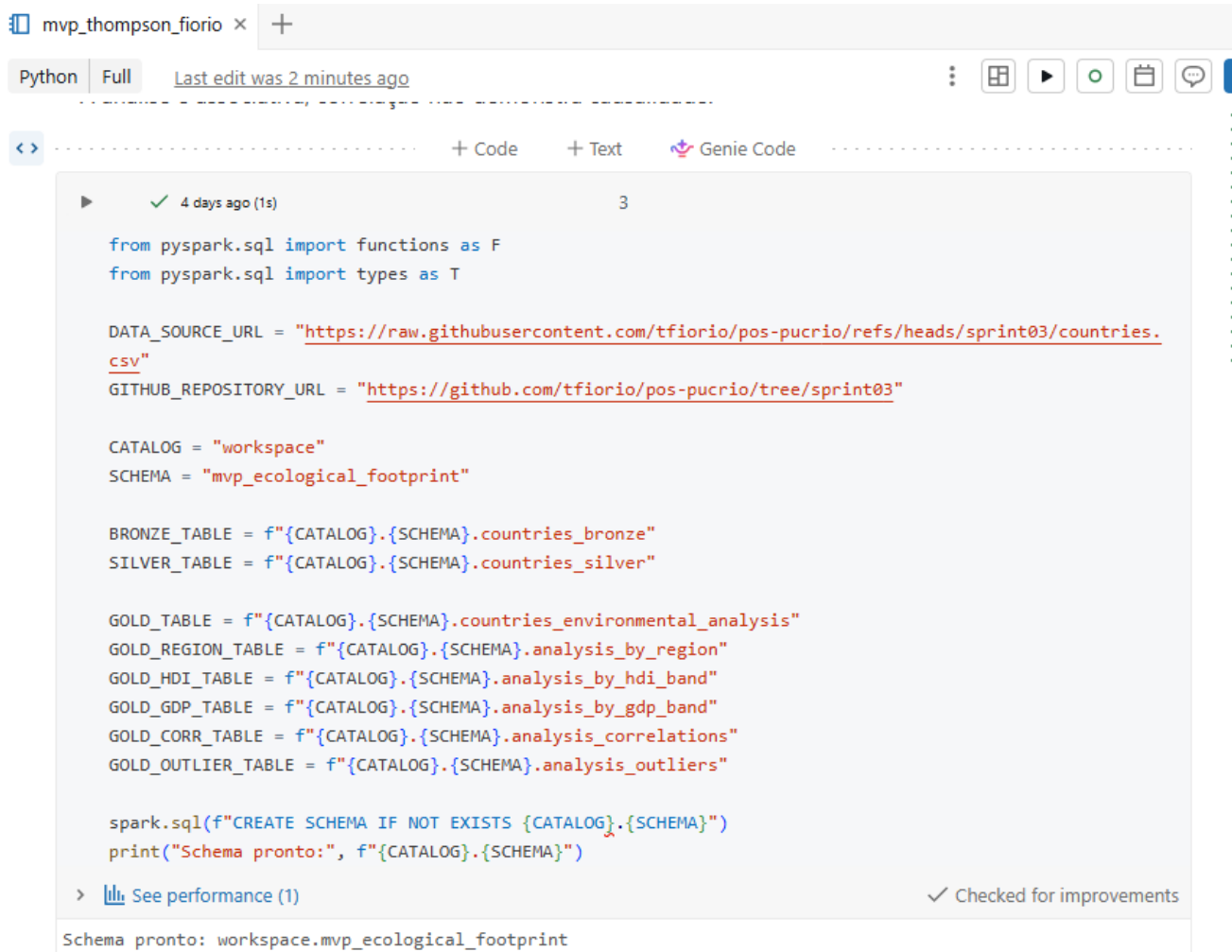
As relações encontradas são interpretadas como associações estatísticas. O estudo não busca estabelecer causalidade entre desenvolvimento socioeconômico e pegada ecológica.

2. Contexto

O desenvolvimento econômico e humano pode estar associado a padrões distintos de consumo de recursos, produção, infraestrutura e uso de energia. Ao mesmo tempo, os componentes da pegada ecológica representam pressões ambientais de naturezas diferentes. Dessa forma, comparar IDH e PIB Per Capita com Pegada de Carbono, Pegada de Peixe, Área de Cultivo e Área de Pastagem permite verificar se o avanço dos indicadores socioeconômicos está associado de maneira uniforme, ou não, às diferentes dimensões ambientais.

A base utilizada no projeto possui 188 registros, correspondentes a países, e 21 atributos originais. Entre os campos estão País, Região, População, IDH, PIB Per Capita, diferentes componentes de Pegada Ecológica, Total de Biocapacidade, Déficit ou Reserva de Biocapacidade, Terras Necessárias e Qualidade dos Dados.

O arquivo utilizado pelo pipeline é obtido a partir do repositório público do projeto no GitHub, branch sprint03. O endereço RAW é configurado no notebook e utilizado como origem da ingestão.



```
from pyspark.sql import functions as F
from pyspark.sql import types as T

DATA_SOURCE_URL = "https://raw.githubusercontent.com/tfiorio/pos-pucrio/refs/heads/sprint03/countries.csv"
GITHUB_REPOSITORY_URL = "https://github.com/tfiorio/pos-pucrio/tree/sprint03"

CATALOG = "workspace"
SCHEMA = "mvp_ecological_footprint"

BRONZE_TABLE = f"{CATALOG}.{SCHEMA}.countries_bronze"
SILVER_TABLE = f"{CATALOG}.{SCHEMA}.countries_silver"

GOLD_TABLE = f"{CATALOG}.{SCHEMA}.countries_environmental_analysis"
GOLD_REGION_TABLE = f"{CATALOG}.{SCHEMA}.analysis_by_region"
GOLD_HDI_TABLE = f"{CATALOG}.{SCHEMA}.analysis_by_hdi_band"
GOLD_GDP_TABLE = f"{CATALOG}.{SCHEMA}.analysis_by_gdp_band"
GOLD_CORR_TABLE = f"{CATALOG}.{SCHEMA}.analysis_correlations"
GOLD_OUTLIER_TABLE = f"{CATALOG}.{SCHEMA}.analysis_outliers"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG}.{SCHEMA}")
print("Schema pronto:", f"{CATALOG}.{SCHEMA}")
```

> [See performance \(1\)](#) ✓ Checked for improvements

Schema pronto: workspace.mvp_ecological_footprint

Figura 01 — Célula de configuração do notebook mostrando DATA_SOURCE_URL, GITHUB_REPOSITORY_URL, CATALOG e SCHEMA.

3. Arquitetura Bronze/Silver/Gold

A solução foi estruturada segundo a arquitetura Medalhão. Essa abordagem separa responsabilidades e permite acompanhar a evolução do dado desde a origem até os produtos analíticos.

Camada	Tabela principal	Finalidade
Bronze	countries_bronze	Preservação dos dados recebidos e inclusão de metadados de ingestão.
Silver	countries_silver	Padronização, tipagem, limpeza e aplicação de regras de qualidade.
Gold	countries_environmental_analysis	Base detalhada oficial para as análises.
Gold auxiliares	analysis_by_region; analysis_by_hdi_band; analysis_by_gdp_band; analysis_correlations; analysis_outliers	Produtos analíticos persistidos para consultas SQL e visualizações.

A decisão de materializar também os resultados auxiliares na camada Gold faz com que a análise seja executada majoritariamente sobre tabelas Delta armazenadas no Databricks, em vez de depender de estruturas locais ou DataFrames temporários.

Table	database	tableName	isTemporary
1	mvp_ecological_footpri...	analysis_by_gdp_band	false
2	mvp_ecological_footpri...	analysis_by_hdi_band	false
3	mvp_ecological_footpri...	analysis_by_region	false
4	mvp_ecological_footpri...	analysis_correlations	false
5	mvp_ecological_footpri...	analysis_outliers	false
6	mvp_ecological_footpri...	countries_bronze	false
7	mvp_ecological_footpri...	countries_environmental_analy...	false
8	mvp_ecological_footpri...	countries_silver	false

Figura 02 — Resultado de `SHOW TABLES IN workspace.mvp_ecological_footprint`, exibindo as oito tabelas persistidas.

4. Coleta e carregamento dos dados

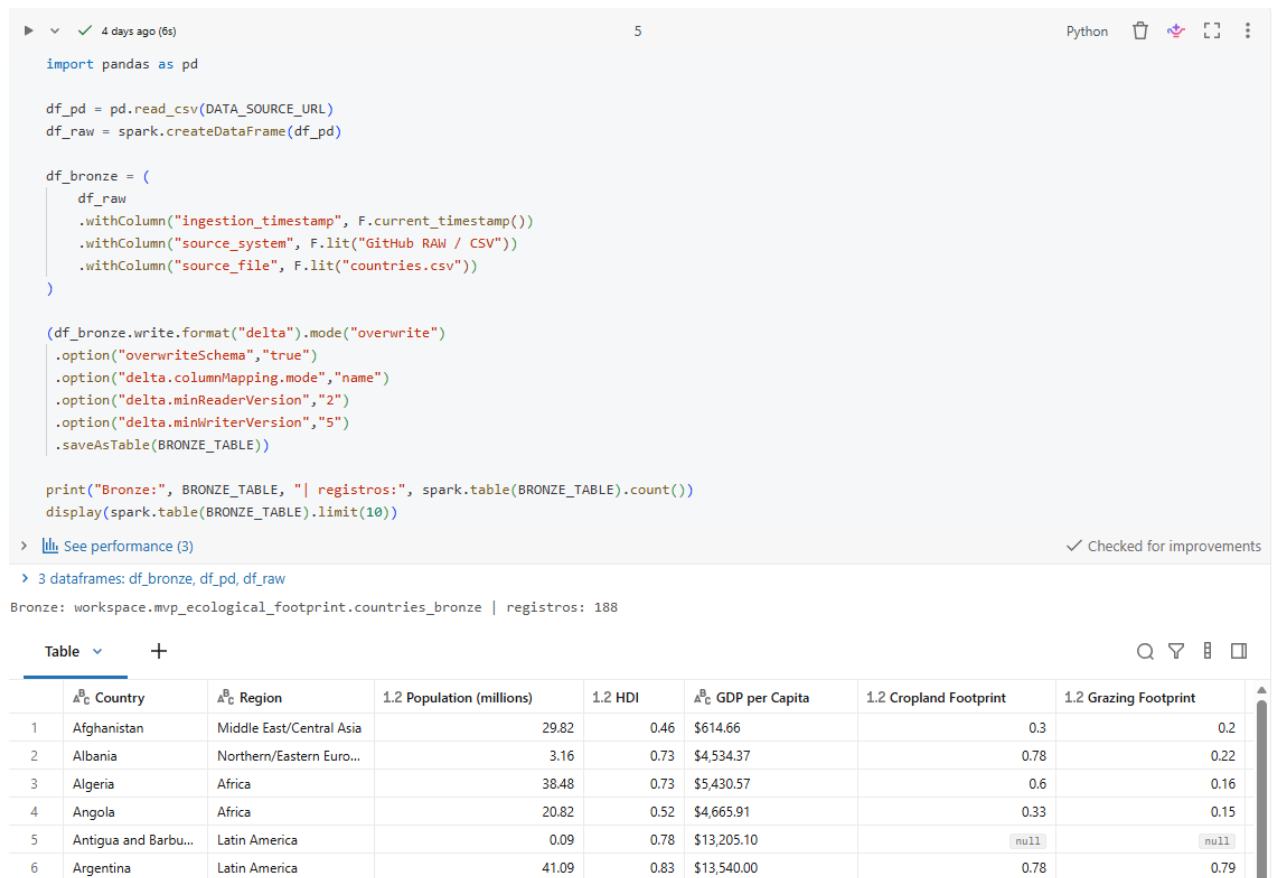
O conjunto de dados utilizado foi originalmente obtido na plataforma KAGGLE, pertencente ao GOOGLE e amplamente utilizada para utilização de *datasets* destinados a projetos de Ciência de Dados. O endereço original do arquivo é [2016 Global Eco](https://www.kaggle.com/datasets/footprintnetwork/ecological-footprint?resource=download) <https://www.kaggle.com/datasets/footprintnetwork/ecological-footprint?resource=download> [logical Footprint](https://www.kaggle.com/datasets/footprintnetwork/ecological-footprint?resource=download). Esse *dataset* é de propriedade da *Global Footprint Network*, e está disponível sob a

licença [CC BY-NC-SA 4.0](#) (*Attribution – NonCommercial – ShareAlike 4.0 International*). Para a realização deste MVP, foi feito download do arquivo CSV no link acima e o publicado na minha página do GitHub.

A coleta, para este MVP, parte do arquivo `countries.csv` publicado no GitHub do projeto. Na execução final, a leitura foi realizada em Python com `pandas.read_csv(DATA_SOURCE_URL)`, seguida da conversão para um `DataFrame Spark` por meio de `spark.createDataFrame`. Essa foi a alteração aplicada na versão executada no Databricks para tornar a ingestão compatível com o ambiente utilizado.

Após a leitura, foram acrescentados os metadados `ingestion_timestamp`, `source_system` e `source_file`. O resultado foi gravado como tabela Delta `countries_bronze` no `schema workspace.mvp_ecological_footprint`. A execução final registrou 188 linhas na camada Bronze.

Para permitir os nomes originais das colunas, que contêm espaços e caracteres especiais, a gravação Delta utiliza *column mapping* por nome e versões mínimas de leitura e escrita compatíveis.



```
import pandas as pd

df_pd = pd.read_csv(DATA_SOURCE_URL)
df_raw = spark.createDataFrame(df_pd)

df_bronze = (
    df_raw
    .withColumn("ingestion_timestamp", F.current_timestamp())
    .withColumn("source_system", F.lit("GitHub RAW / CSV"))
    .withColumn("source_file", F.lit("countries.csv"))
)

(df_bronze.write.format("delta").mode("overwrite")
 .option("overwriteSchema", "true")
 .option("delta.columnMapping.mode", "name")
 .option("delta.minReaderVersion", "2")
 .option("delta.minWriterVersion", "5")
 .saveAsTable(BRONZE_TABLE))

print("Bronze:", BRONZE_TABLE, "| registros:", spark.table(BRONZE_TABLE).count())
display(spark.table(BRONZE_TABLE).limit(10))
```

See performance (3) ✓ Checked for improvements

3 dataframes: df_bronze, df_pd, df_raw

Bronze: workspace.mvp_ecological_footprint.countries_bronze | registros: 188

	Country	Region	1.2 Population (millions)	1.2 HDI	1.2 GDP per Capita	1.2 Cropland Footprint	1.2 Grazing Footprint
1	Afghanistan	Middle East/Central Asia	29.82	0.46	\$614.66	0.3	0.2
2	Albania	Northern/Eastern Euro...	3.16	0.73	\$4,534.37	0.78	0.22
3	Algeria	Africa	38.48	0.73	\$5,430.57	0.6	0.16
4	Angola	Africa	20.82	0.52	\$4,665.91	0.33	0.15
5	Antigua and Barbu...	Latin America	0.09	0.78	\$13,205.10	null	null
6	Argentina	Latin America	41.09	0.83	\$13,540.00	0.78	0.79

Figura 03 — Célula que cria `df_bronze` e a saída 'Bronze: workspace.mvp_ecological_footprint.countries_bronze | registros: 188', incluindo parte da tabela exibida.

5. Modelagem de dados

O conjunto de dados possui granularidade de um registro por país. Como as perguntas de análise utilizam atributos associados à mesma entidade analítica, foi adotado um modelo flat na camada Gold, evitando a criação artificial de dimensões e fatos sem ganho para o escopo do MVP.

A camada Silver é derivada da Bronze persistida. Nessa etapa, os nomes das colunas são padronizados para *snake_case*, os atributos numéricos são convertidos explicitamente para *double* e o *GDP Per Capita*, originalmente representado como texto monetário, é transformado em valor numérico após a remoção dos símbolos '\$' e das vírgulas.

Também são removidas duplicidades integrais e criados indicadores de qualidade. O campo *is_complete_for_analysis* identifica os registros que possuem valores válidos nas seis variáveis centrais: *HDI*, *GDP Per Capita*, *Carbon Footprint*, *Fish Footprint*, *Cropland Footprint* e *Grazing Footprint*.

A Gold principal seleciona somente os registros completos e válidos, preservando país, região, população e as variáveis centrais. Também são criadas faixas de IDH e PIB Per Capita para apoiar análises agregadas. O resultado final contém 162 países.

```

4 days ago (8s) 9 Python
rename_map = {
    "Country": "country", "Region": "region", "Population (millions)": "population_millions",
    "HDI": "hdi", "GDP per Capita": "gdp_per_capita_raw",
    "Cropland Footprint": "cropland_footprint", "Grazing Footprint": "grazing_footprint",
    "Forest Footprint": "forest_footprint", "Carbon Footprint": "carbon_footprint",
    "Fish Footprint": "fish_footprint", "Total Ecological Footprint": "total_ecological_footprint",
    "Cropland": "cropland", "Grazing Land": "grazing_land", "Forest Land": "forest_land",
    "Fishing Water": "fishing_water", "Urban Land": "urban_land",
    "Total Biocapacity": "total_biocapacity",
    "Biocapacity Deficit or Reserve": "biocapacity_deficit_or_reserve",
    "Earths Required": "earths_required", "Countries Required": "countries_required",
    "Data Quality": "data_quality"
}

silver = spark.table(BRONZE_TABLE)
for old,new in rename_map.items():
    silver = silver.withColumnRenamed(old,new)

silver = silver.withColumn(
    "gdp_per_capita",
    F.regexp_replace(F.col("gdp_per_capita_raw").cast("string"), r"[\$,]", "").cast("double")
)

numeric_cols = [
    "population_millions", "hdi", "cropland_footprint", "grazing_footprint",
    "forest_footprint", "carbon_footprint", "fish_footprint",
    "total_ecological_footprint", "cropland", "grazing_land", "forest_land",
    "fishing_water", "urban_land", "total_biocapacity",
    "biocapacity_deficit_or_reserve", "earths_required", "countries_required"
]
for c in numeric_cols:
    silver = silver.withColumn(c, F.col(c).cast("double"))

analysis_cols = ["hdi", "gdp_per_capita", "carbon_footprint", "fish_footprint",
                 "cropland_footprint", "grazing_footprint"]
complete = F.lit(True)
for c in analysis_cols:
    complete = complete & F.col(c).isNotNull()

silver = (
    silver.dropDuplicates()
    .withColumn("is_complete_for_analysis", complete)
    .withColumn("quality_rule_hdi",
        F.when(F.col("hdi").isNull(), "MISSING")
        .when((F.col("hdi") < 0) | (F.col("hdi") > 1), "INVALID").otherwise("OK"))
    .withColumn("quality_rule_footprints",
        F.when((F.col("carbon_footprint") < 0) | (F.col("fish_footprint") < 0) |
            (F.col("cropland_footprint") < 0) | (F.col("grazing_footprint") < 0),
            "INVALID").otherwise("OK"))
    .withColumn("silver_processed_timestamp", F.current_timestamp())
)

(silver.write.format("delta").mode("overwrite")
    .option("overwriteSchema", "true").saveAsTable(SILVER_TABLE))

print("Silver:", SILVER_TABLE, "| registros:", spark.table(SILVER_TABLE).count())

```

See performance (2) ✓ Checked for improvements

silver: pyspark.sql.connect.dataframe.DataFrame = [country: string, region: string ... 27 more fields]

Silver: workspace.mvp_ecological_footprint.countries_silver | registros: 188

Figura 04 — Célula que mostra a conversão de `gdp_per_capita`, as regras `is_complete_for_analysis/quality_rule` e a confirmação de 188 registros Silver.

4 days ago (6s)

13

Python

```

gold = (
    spark.table(SILVER_TABLE)
    .filter("is_complete_for_analysis = true")
    .filter("quality_rule_hdi = 'OK'")
    .filter("quality_rule_footprints = 'OK'")
    .select("country", "region", "population_millions", "hdi", "gdp_per_capita",
           "carbon_footprint", "fish_footprint", "cropland_footprint",
           "grazing_footprint", "data_quality")
    .withColumn("hdi_band",
               F.when(F.col("hdi") < .550, "Baixo")
                 .when(F.col("hdi") < .700, "Médio")
                 .when(F.col("hdi") < .800, "Alto").otherwise("Muito alto"))
    .withColumn("gdp_per_capita_band",
               F.when(F.col("gdp_per_capita") < 2000, "< 2 mil")
                 .when(F.col("gdp_per_capita") < 10000, "2-10 mil")
                 .when(F.col("gdp_per_capita") < 30000, "10-30 mil").otherwise(">= 30 mil"))
    .withColumn("gold_processed_timestamp", F.current_timestamp())
)

(gold.write.format("delta").mode("overwrite")
 .option("overwriteSchema", "true").saveAsTable(GOLD_TABLE))

print("Gold principal:", GOLD_TABLE, "| registros:", spark.table(GOLD_TABLE).count())
display(spark.table(GOLD_TABLE).limit(10))

```

[See performance \(3\)](#)

✓ Checked for improvements

gold: pyspark.sql.connect.dataframe.DataFrame = [country: string, region: string ... 11 more fields]

Gold principal: workspace.mvp_ecological_footprint.countries_environmental_analysis | registros: 162

Table +

1.2 country

1.2 region

1.2 population_millions

1.2 hdi

1.2 gdp_per_capita

1.2 carbon_footprint

1.2 fish_footprint

1.2 cr

1	Albania	Northern/Eastern Euro...	3.16	0.73	4534.37	0.87	0.02	
2	Bosnia and Herzegovi...	Northern/Eastern Euro...	3.83	0.73	4708.85	1.74	0.02	
3	Dominica	Latin America	0.07	0.72	7088.01	1.47	0.17	
4	Georgia	Middle East/Central Asia	4.36	0.75	3710.7	0.83	0.02	
5	Libyan Arab Jamahiriya	Africa	6.16	0.74	5859.43	2.33	0.1	

Figura 05 — Célula de criação de `countries_environmental_analysis` e a saída indicando 162 registros, com algumas linhas da tabela.

6. Catálogo de dados

O catálogo abaixo descreve os principais atributos utilizados na camada Gold. A linhagem segue o fluxo `countries.csv` → `countries_bronze` → `countries_silver` → `countries_environmental_analysis`.

9

Campo	Tipo	Descrição
country	string	Nome do país.
region	string	Região geográfica associada ao país.
population_millions	double	População em milhões.
hdi	double	Índice de Desenvolvimento Humano (IDH).
gdp_per_capita	double	PIB Per Capita após conversão do campo monetário original.
carbon_footprint	double	Componente da pegada ecológica relacionado a carbono.
fish_footprint	double	Componente da pegada ecológica relacionado à pesca.
cropland_footprint	double	Componente da pegada ecológica relacionado a áreas de cultivo.
grazing_footprint	double	Componente da pegada ecológica relacionado a pastagens.
data_quality	string	Indicador de qualidade presente na fonte.
hdi_band	string	Faixa de IDH criada na camada Gold.
gdp_per_capita_band	string	Faixa de PIB Per Capita criada na camada Gold.
gold_processed_timestamp	timestamp	Data e hora de processamento da Gold.

Além da Gold detalhada, foram persistidas tabelas auxiliares para análise por região, faixa de IDH, faixa de PIB, correlações e *outliers*. Essa materialização facilita a reutilização das informações por SQL e por visualizações do Databricks.

print("Gold principal:", GOLD_TABLE, " | registros:", spark.table(GOLD_TABLE).count())
display(spark.table(GOLD_TABLE).limit(10))

Gold principal: workspace.mvp_ecological_footprint.countries_environmental_analysis | registros: 162

	country	region	1.2 population_millions	1.2 hdi	1.2 gdp_per_capita	1.2 carbon_footprint	1.2 fish_footprint
1	Albania	Northern/Eastern Euro...	3.16	0.73	4534.37	0.87	0
2	Bosnia and Herzegovina	Northern/Eastern Euro...	3.83	0.73	4708.85	1.74	0
3	Dominica	Latin America	0.07	0.72	7088.01	1.47	0
4	Georgia	Middle East/Central Asia	4.36	0.75	3710.7	0.83	0
5	Libyan Arab Jamahiriya	Africa	6.16	0.74	5859.43	2.33	0
6	Mali	Africa	14.85	0.41	741.22	0.1	0
7	Netherlands	European Union	16.71	0.92	53589.9	3.23	0
8	Qatar	Middle East/Central Asia	2.05	0.85	99431.5	9.57	0
9	Sao Tome and Principe	Africa	0.19	0.55	1384.53	0.37	0
10							

10 rows | 6.004s runtime Refreshed 4d ago

Figura 06 — Catalog Explorer/Unity Catalog o schema *mvp_ecological_footprint* e as colunas e alguns dados da tabela *countries_environmental_analysis*.

7. Pipeline de dados

O pipeline pode ser representado pelo fluxo: GitHub/CSV → Bronze → Silver → Gold detalhada → Gold auxiliares → consultas SQL e visualizações. Cada etapa possui uma responsabilidade específica e utiliza como entrada uma estrutura persistida da etapa anterior.

Na Bronze, o dado é mantido próximo ao formato recebido. Na Silver, são aplicadas padronização, tipagem e regras de qualidade. Na Gold, os registros completos são disponibilizados para consumo analítico. Por fim, as tabelas auxiliares materializam agregações e cálculos que seriam reutilizados nas consultas finais.

Foram criadas as tabelas *analysis_by_region*, *analysis_by_hdi_band*, *analysis_by_gdp_band*, *analysis_correlations* e *analysis_outliers*. Dessa forma, as análises não dependem da repetição de todo o processo de preparação.

4 days ago (11s)

15

Python



```
spark.sql(f'''
CREATE OR REPLACE TABLE {GOLD_REGION_TABLE}
USING DELTA AS
SELECT
    region,
    COUNT(*) AS countries,
    ROUND(AVG(hdi),3) AS avg_hdi,
    ROUND(AVG(gdp_per_capita),2) AS avg_gdp_per_capita,
    ROUND(AVG(carbon_footprint),3) AS avg_carbon_footprint,
    ROUND(AVG(fish_footprint),3) AS avg_fish_footprint,
    ROUND(AVG(cropland_footprint),3) AS avg_cropland_footprint,
    ROUND(AVG(grazing_footprint),3) AS avg_grazing_footprint
FROM {GOLD_TABLE}
GROUP BY region
''')

spark.sql(f'''
CREATE OR REPLACE TABLE {GOLD_HDI_TABLE}
USING DELTA AS
SELECT
    hdi_band,
    COUNT(*) AS countries,
    ROUND(AVG(hdi),3) AS avg_hdi,
    ROUND(AVG(gdp_per_capita),2) AS avg_gdp_per_capita,
    ROUND(AVG(carbon_footprint),3) AS avg_carbon_footprint,
    ROUND(AVG(fish_footprint),3) AS avg_fish_footprint,
    ROUND(AVG(cropland_footprint),3) AS avg_cropland_footprint,
    ROUND(AVG(grazing_footprint),3) AS avg_grazing_footprint
FROM {GOLD_TABLE}
GROUP BY hdi_band
''')

spark.sql(f'''
CREATE OR REPLACE TABLE {GOLD_GDP_TABLE}
USING DELTA AS
SELECT
    gdp_per_capita_band,
    COUNT(*) AS countries,
    ROUND(AVG(hdi),3) AS avg_hdi,
    ROUND(AVG(gdp_per_capita),2) AS avg_gdp_per_capita,
    ROUND(AVG(carbon_footprint),3) AS avg_carbon_footprint,
    ROUND(AVG(fish_footprint),3) AS avg_fish_footprint,
    ROUND(AVG(cropland_footprint),3) AS avg_cropland_footprint,
    ROUND(AVG(grazing_footprint),3) AS avg_grazing_footprint
FROM {GOLD_TABLE}
GROUP BY gdp_per_capita_band
''')

print("Tabelas Gold agregadas criadas.")
```

[See performance \(3\)](#)

✓ Checked for improvements

Tabelas Gold agregadas criadas.

Figura 07 — Célula que executa `CREATE OR REPLACE TABLE` para as tabelas por região, faixa de HDI e faixa de PIB e a mensagem 'Tabelas Gold agregadas criadas'.

8. Qualidade dos dados

A avaliação inicial foi executada sobre a própria tabela Bronze persistida. A base possui 188 registros, sem duplicidades integrais e sem países duplicados. Foram encontrados 16 valores ausentes em *HDI* e 15 valores ausentes em *GDP Per Capita*, *Carbon Footprint*, *Fish Footprint*, *Cropland Footprint* e *Grazing Footprint*.

Na camada Silver, 162 registros foram classificados como completos para análise. As regras de qualidade indicaram 172 registros com *HDI* válido e *footprints* válidos, enquanto 16 registros apresentaram *HDI* ausente. Os registros incompletos são preservados na Silver para manter rastreabilidade; a filtragem ocorre apenas na construção da Gold analítica.

Também foram aplicadas verificações de domínio: *HDI* deve estar entre 0 e 1, e os quatro *footprints* analisados não podem assumir valores negativos. Essa estratégia evita imputar valores artificiais em indicadores de países e reduz o risco de introduzir viés nas correlações.

```

bronze = spark.table(BRONZE_TABLE)

null_exprs = [
    F.sum(F.when(F.col(c).isNull() | (F.trim(F.col(c).cast("string")) == ""), 1).otherwise(0)).alias(c)
    for c in df_raw.columns
]
display(bronze.select(null_exprs))

print("Registros Bronze:", bronze.count())
print("Duplicidades integrais:",
      bronze.groupBy(*df_raw.columns).count().filter("count > 1").count())
print("Países duplicados:",
      bronze.groupBy("Country").count().filter("count > 1").count())

```

	Country	Region	Population (millions)	HDI	GDP per Capita	Cropland Footprint	Grazing Footprint
1	0	0	0	16	15	15	15

Figura 08 — Consulta da qualidade inicial da Bronze: contagem de nulos, 188 registros, 0 duplicidades integrais e 0 países duplicados.

```

display(spark.sql(f'''
SELECT
    COUNT(*) AS total_registros,
    SUM(CASE WHEN hdi IS NULL THEN 1 ELSE 0 END) AS hdi_nulos,
    SUM(CASE WHEN gdp_per_capita IS NULL THEN 1 ELSE 0 END) AS gdp_nulos,
    SUM(CASE WHEN carbon_footprint IS NULL THEN 1 ELSE 0 END) AS carbon_nulos,
    SUM(CASE WHEN fish_footprint IS NULL THEN 1 ELSE 0 END) AS fish_nulos,
    SUM(CASE WHEN cropland_footprint IS NULL THEN 1 ELSE 0 END) AS cropland_nulos,
    SUM(CASE WHEN grazing_footprint IS NULL THEN 1 ELSE 0 END) AS grazing_nulos,
    SUM(CASE WHEN is_complete_for_analysis THEN 1 ELSE 0 END) AS completos_analise
FROM {SILVER_TABLE}
'''))

display(spark.sql(f'''
SELECT quality_rule_hdi, quality_rule_footprints, COUNT(*) AS registros
FROM {SILVER_TABLE}
GROUP BY quality_rule_hdi, quality_rule_footprints
ORDER BY registros DESC
'''))

```

	total_registros	hdi_nulos	gdp_nulos	carbon_nulos	fish_nulos	cropland_nulos	grazing_nulos	completos_analise
1	188	16	15	15	15	15	15	15

Figura 09 — Consulta SQL da Silver mostrando 188 registros, 16 HDI nulos, 15 nulos nas demais variáveis centrais e 162 registros completos.

9. Análises

A etapa analítica utiliza principalmente Spark SQL sobre tabelas Delta persistidas. O coeficiente de Pearson foi calculado pelo Spark sobre a Gold principal e armazenado em *analysis_correlations*. As agregações regionais e por faixas também foram materializadas em tabelas Gold específicas.

9.1 Correlações entre desenvolvimento e pegadas

Relação	Correlação de Pearson
PIB Per Capita × Pegada de Carbono	0,824
IDH × Pegada de Carbono	0,699
IDH × Área de Cultivo	0,567
PIB Per Capita × Área de Cultivo	0,506
IDH × Pegada de Pesca	0,208
PIB Per Capita × Pegada de Pesca	0,153
PIB Per Capita × Área de Pastagem	0,103
IDH × Área de Pastagem	0,091

A associação mais intensa foi observada entre PIB Per Capita e Pegada de Carbono (0,824). A relação entre IDH e Pegada de Carbono também foi positiva e elevada (0,699). Área de Cultivo apresentou associações intermediárias com IDH (0,567) e PIB Per Capita (0,506). Já Pegada de Peixe e Área de Pastagem apresentaram correlações positivas, porém de baixa intensidade.

```

# 8.1 Relações entre desenvolvimento e footprints
display(spark.sql(f'''
SELECT
  variable_x,
  variable_y,
  ROUND(correlation, 3) AS pearson_correlation
FROM {GOLD_CORR_TABLE}
ORDER BY ABS(correlation) DESC
'''))

```

	variable_x	variable_y	pearson_correlation
1	gdp_per_capita	carbon_footprint	0.824
2	hdi	carbon_footprint	0.699
3	hdi	cropland_footprint	0.567
4	gdp_per_capita	cropland_footprint	0.506
5	hdi	fish_footprint	0.208
6	gdp_per_capita	fish_footprint	0.153
7	gdp_per_capita	grazing_footprint	0.103
8	hdi	grazing_footprint	0.091

8 rows | 1.073s runtime | Refreshed 4d ago

Figura 10 — Resultado da consulta à tabela `analysis_correlations`, ordenado pelo valor absoluto da correlação.

9.2 Análise por região

Região	Países	IDH médio	PIB pc médio	Carbono	Peixe	Área de Cultivo	Área de Pastagem
North America	2	0,910	50.935,20	5,450	0,120	1,295	0,315
European Union	24	0,865	35.959,72	3,264	0,116	0,983	0,260
Northern/Eastern Europe	11	0,775	14.209,65	2,211	0,060	0,708	0,160
Middle East/Central Asia	22	0,732	16.368,72	3,046	0,067	0,602	0,217
Latin America	29	0,718	8.319,64	1,618	0,131	0,489	0,414
Asia-Pacific	27	0,693	12.611,26	1,503	0,215	0,602	0,248
Africa	47	0,514	2.705,44	0,541	0,095	0,397	0,234

A América do Norte apresentou os maiores valores médios de IDH (0,910), PIB Per Capita (50.935,20) e Pegada de Carbono (5,450), embora a região possua apenas dois países na amostra analítica. A União Europeia apresentou IDH médio de 0,865 e Pegada de Carbono média de 3,264. Em contraste, a África apresentou IDH médio de 0,514, PIB Per Capita médio de 2.705,44 e Pegada de Carbono média de 0,541.

Os resultados regionais reforçam a associação observada nas correlações, especialmente para carbono, mas também demonstram que os demais componentes possuem comportamentos distintos entre regiões.

```

# 8.2 Perfil por região
display(spark.sql(f'''
SELECT *
FROM GOLD_REGION_TABLE
ORDER BY avg_hdi DESC
'''))

```

	region	countries	avg_hdi	avg_gdp_per_capita	avg_carbon_footprint	avg_fish_footprint	avg_cropl
1	North America	2	0.91	50935.2	5.45	0.12	
2	European Union	24	0.865	35959.72	3.264	0.116	
3	Northern/Eastern Euro...	11	0.775	14209.65	2.211	0.06	
4	Middle East/Central Asia	22	0.732	16368.72	3.046	0.067	
5	Latin America	29	0.718	8319.64	1.618	0.131	
6	Asia-Pacific	27	0.693	12611.26	1.503	0.215	
7							

7 rows | 1.222s runtime | Refreshed 4d ago

Figura 11 — Resultado consulta SQL à tabela *analysis_by_region* ordenada por *avg_hdi*.

9.3 Análise por faixa de IDH

Faixa IDH	Países	IDH médio	PIB pc médio	Carbono	Peixe	Área de Cultivo	Área de Pastagem
Baixo	40	0,461	1.035,68	0,202	0,077	0,368	0,219
Médio	32	0,629	3.932,62	0,951	0,092	0,456	0,231
Alto	48	0,747	8.164,54	1,927	0,147	0,578	0,318
Muito alto	42	0,867	38.835,29	3,869	0,146	0,936	0,279

A Pegada de Carbono média cresce de 0,202 na faixa de IDH baixo para 3,869 na faixa de IDH muito alto. A Área de Cultivo também apresenta crescimento progressivo, de 0,368 para 0,936. Pegada de Peixe e Área de Pastagem não exibem a mesma progressão clara, evidenciando que a relação entre desenvolvimento e pressão ambiental depende do componente analisado.

4 days ago (2s) 21 Python

```
# 8.3 Perfil por faixa de HDI
display(spark.sql(f'''
SELECT *
FROM {GOLD_HDI_TABLE}
ORDER BY
CASE hdi_band
WHEN 'Baixo' THEN 1 WHEN 'Médio' THEN 2
WHEN 'Alto' THEN 3 WHEN 'Muito alto' THEN 4
END
'''))
```

See performance (1) ✓ Checked for improvements

	hdi_band	countries	1.2 avg_hdi	1.2 avg_gdp_per_capita	1.2 avg_carbon_footprint	1.2 avg_fish_footprint	1.2 avg_cropland_foo
1	Baixo	40	0.461	1035.68	0.202	0.077	
2	Médio	32	0.629	3932.62	0.951	0.092	
3	Alto	48	0.747	8164.54	1.927	0.147	
4	Muito alto	42	0.867	38835.29	3.869	0.146	

Figura 12 — Resultado da consulta à tabela analysis_by_hdi_band.

9.4 Análise por faixa de PIB Per Capita

Faixa PIB pc	Países	IDH médio	PIB pc médio	Carbono	Peixe	Área de Cultivo	Área de Pastagem
< 2 mil	50	0,496	968,79	0,285	0,077	0,379	0,184
2–10 mil	57	0,701	5.313,10	1,331	0,114	0,552	0,299
10–30 mil	30	0,796	16.864,54	2,970	0,156	0,726	0,307
>= 30 mil	25	0,889	53.321,15	4,573	0,167	0,966	0,306

A segmentação por PIB Per Capita apresenta tendência semelhante para Pegada de Carbono: a média cresce de 0,285 no grupo abaixo de 2 mil para 4,573 no grupo igual ou superior a 30 mil. Área de Cultivo também cresce entre as faixas. Pegada de Peixe apresenta crescimento menor, enquanto Área de Pastagem aumenta nas faixas intermediárias e praticamente se estabiliza nas duas faixas superiores.

4 days ago (1s) 22

```
# 8.4 Perfil por faixa de PIB per capita
display(spark.sql(f'''
SELECT *
FROM {GOLD_GDP_TABLE}
ORDER BY avg_gdp_per_capita
'''))
```

See performance (1) ✓ Checked for improvements

	1.0 gdp_per_capita_band	1.2 countries	1.2 avg_hdi	1.2 avg_gdp_per_capita	1.2 avg_carbon_footprint	1.2 avg_fish_footprint	1.2 avg_c
1	< 2 mil	50	0.496	968.79	0.285	0.077	
2	2-10 mil	57	0.701	5313.1	1.331	0.114	
3	10-30 mil	30	0.796	16864.54	2.97	0.156	
4	>= 30 mil	25	0.889	53321.15	4.573	0.167	

4 rows | 1.377s runtime Refreshed 4d ago

Figura 13 — Resultado da consulta à tabela *analysis_by_gdp_band*.

9.5 Países com maiores valores

Na análise de Pegada de Carbono, Luxemburgo apresentou o maior valor (12,65), seguido por Qatar (9,57). Trinidad e Tobago e Kuwait apresentaram 6,89, Emirados Árabes Unidos 6,37 e Bahrain 6,19. Esses países também aparecem na análise de outliers de carbono.

Para Pegada de Peixe, os maiores valores foram observados em Saint Kitts and Nevis (0,81), Papua New Guinea (0,73) e Namibia (0,72). Para Área de Cultivo, Australia (2,68), Latvia (2,28) e Lithuania (1,89) ocuparam as primeiras posições. Em Área de Pastagem, Mongolia apresentou valor de 3,47, bastante acima de Bolívia (1,69) e Mauritania (1,20).

4 days ago (1s) 23 Python

```
# 8.5 Países com maior Carbon Footprint
display(spark.sql(f'''
SELECT country, region, hdi, gdp_per_capita, carbon_footprint
FROM {GOLD_TABLE}
ORDER BY carbon_footprint DESC
LIMIT 20
'''))
```

See performance (1) ✓ Checked for improvements

	country	region	1.2 hdi	1.2 gdp_per_capita	1.2 carbon_footprint
1	Luxembourg	European Union	0.89	114665	12.65
2	Qatar	Middle East/Central Asia	0.85	99431.5	9.57
3	Trinidad and Tobago	Latin America	0.77	18310.8	6.89
4	Kuwait	Middle East/Central Asia	0.82	41830.5	6.89
5	United Arab Emirates	Middle East/Central Asia	0.83	40817.4	6.37
6	Bahrain	Middle East/Central Asia	0.82	24299	6.19
7	Singapore	Asia-Pacific	0.91	53122.4	5.91
8	United States of America	North America	0.91	49725	5.9
9	Oman	Middle East/Central Asia	0.79	22622.8	5.8
10	Canada	North America	0.91	52145.4	5
11	Australia	Asia-Pacific	0.93	66604.2	4.85
12	Israel	Middle East/Central Asia	0.89	33718.9	4.59
13	Kazakhstan	Middle East/Central Asia	0.78	11278	4.59
14	Bahamas	Latin America	0.78	22647.3	4.46
15	Belgium	European Union	0.89	48053.3	4.43

20 rows | 1.028s runtime Refreshed 4d ago

Figura — Consulta dos 20 países com maior Pegada de Carbono

▶

✓ 4 days ago (3s)

24

Python

```
# 8.6. Países com maior Fish Cropland e Grazing Footprint
Click to add a breakpoint.
SELECT country, region, fish_footprint
FROM {GOLD_TABLE}
ORDER BY fish_footprint DESC
LIMIT 15
'''))

display(spark.sql(f'''
SELECT country, region, cropland_footprint
FROM {GOLD_TABLE}
ORDER BY cropland_footprint DESC
LIMIT 15
'''))

display(spark.sql(f'''
SELECT country, region, grazing_footprint
FROM {GOLD_TABLE}
ORDER BY grazing_footprint DESC
LIMIT 15
'''))
```

> [See performance \(3\)](#)

✓ Checked for improvements

Table ▾ +

	country	region	1.2 fish_footprint
1	Saint Kitts and Nevis	Latin America	0.81
2	Papua New Guinea	Asia-Pacific	0.73
3	Namibia	Africa	0.72
4	New Zealand	Asia-Pacific	0.7
5	Fiji	Asia-Pacific	0.62
6	Mauritius	Africa	0.55
7	Sao Tome and Principe	Africa	0.47
8	Solomon Islands	Asia-Pacific	0.46
9	Samoa	Asia-Pacific	0.42
10	Korea, Republic of	Asia-Pacific	0.41
11	Oman	Middle East/Central As...	0.4
12	Panama	Latin America	0.37
13	Malaysia	Asia-Pacific	0.36
14	Japan	Asia-Pacific	0.33
15	Peru	Latin America	0.33

15 rows

3.376s runtime

Refreshed 4d ago

Table ▾ +				Q	Y	⌵	⌶
	^A _C country	^A _C region	1.2 cropland_footprint				
1	Australia	Asia-Pacific	2.68				
2	Latvia	European Union	2.28				
3	Lithuania	European Union	1.89				
4	Sweden	European Union	1.47				
5	Canada	North America	1.46				
6	Belarus	Northern/Eastern Euro...	1.32				
7	France	European Union	1.23				
8	Tonga	Asia-Pacific	1.19				
9	Denmark	European Union	1.18				
10	Belgium	European Union	1.15				
11	United States of America	North America	1.13				
12	Paraguay	Latin America	1.11				
13	Germany	European Union	1.1				
14	Luxembourg	European Union	1.1				
15	Ireland	European Union	1.08				

▾ 15 rows 3.376s runtime
 Refreshed 4d ago

Table ▾ +				Q	Y	⌵	⌶
	^A _C country	^A _C region	1.2 grazing_footprint				
1	Mongolia	Asia-Pacific	3.47				
2	Bolivia	Latin America	1.69				
3	Mauritania	Africa	1.2				
4	Paraguay	Latin America	1.1				
5	Bahamas	Latin America	1.05				
6	Uruguay	Latin America	0.98				
7	Botswana	Africa	0.89				
8	Brazil	Latin America	0.85				
9	Argentina	Latin America	0.79				
10	Luxembourg	European Union	0.76				
11	Chad	Africa	0.75				
12	Venezuela, Bolivarian Republic of	Latin America	0.74				
13	Mali	Africa	0.66				
14	Australia	Asia-Pacific	0.63				
15	Colombia	Latin America	0.58				

▾ 15 rows 3.376s runtime
 Refreshed 4d ago

Figura 15 — Consultas de Pegada de Pesca, Cultivo e Pastagem

9.6 Outliers

Os *outliers* foram identificados pelo método do intervalo interquartil (IQR) e armazenados em *analysis_outliers*. Foram encontrados 23 *outliers* de PIB Per Capita, 15 de Pegada de Peixe, 12 de Área de Pastagem, 6 de Pegada de Carbono e 6 de Pegada de Cultivo.

O uso do IQR funciona como sinalizador de observações extremas, e não como regra automática de exclusão. Em um estudo entre países, valores elevados podem representar características econômicas, energéticas, produtivas ou territoriais legítimas e, portanto, são relevantes para a interpretação.

The screenshot shows a Databricks notebook interface. At the top, there's a status bar with a play button, a dropdown menu, a checkmark, the text '4 days ago (3s)', a page number '25', and icons for Python, trash, and other actions. The main area contains two SQL queries. The first query is a comment '# 8.7 Outliers persistidos' followed by a display statement and a SQL query that selects 'metric' and 'COUNT(*) AS outlier_count' from 'GOLD_OUTLIER_TABLE', grouped by 'metric' and ordered by 'outlier_count' in descending order. The second query is another display statement and a SQL query that selects all columns from 'GOLD_OUTLIER_TABLE' and orders them by 'metric' and 'metric_value' in descending order. Below the queries, there's a link 'See performance (2)' and a checkmark indicating it's 'Checked for improvements'. The bottom section shows a table with two columns: 'metric' and 'outlier_count'. The table has 5 rows of data. At the bottom of the table, there's a download icon, a dropdown menu, the text '5 rows | 3.052s runtime', and a 'Refreshed 4d ago' status.

```
# 8.7 Outliers persistidos
display(spark.sql(f'''
SELECT metric, COUNT(*) AS outlier_count
FROM {GOLD_OUTLIER_TABLE}
GROUP BY metric
ORDER BY outlier_count DESC
'''))

display(spark.sql(f'''
SELECT *
FROM {GOLD_OUTLIER_TABLE}
ORDER BY metric, metric_value DESC
'''))
```

> [See performance \(2\)](#) ✓ Checked for improvements

	metric	outlier_count
1	gdp_per_capita	23
2	fish_footprint	15
3	grazing_footprint	12
4	carbon_footprint	6
5	cropland_footpri...	6

5 rows | 3.052s runtime Refreshed 4d ago

Figura 16 — Consulta com a quantidade de outliers por métrica e parte da tabela analysis_outliers.

9.7 Visualizações recomendadas no Databricks

Para manter coerência com a proposta do MVP, os gráficos foram gerados a partir das consultas às tabelas Gold no próprio Databricks. Os dois gráficos de dispersão mais importantes são IDH × Pegada de Carbono e PIB Per Capita × Pegada de Carbono, pois representam as associações de maior interesse do estudo.

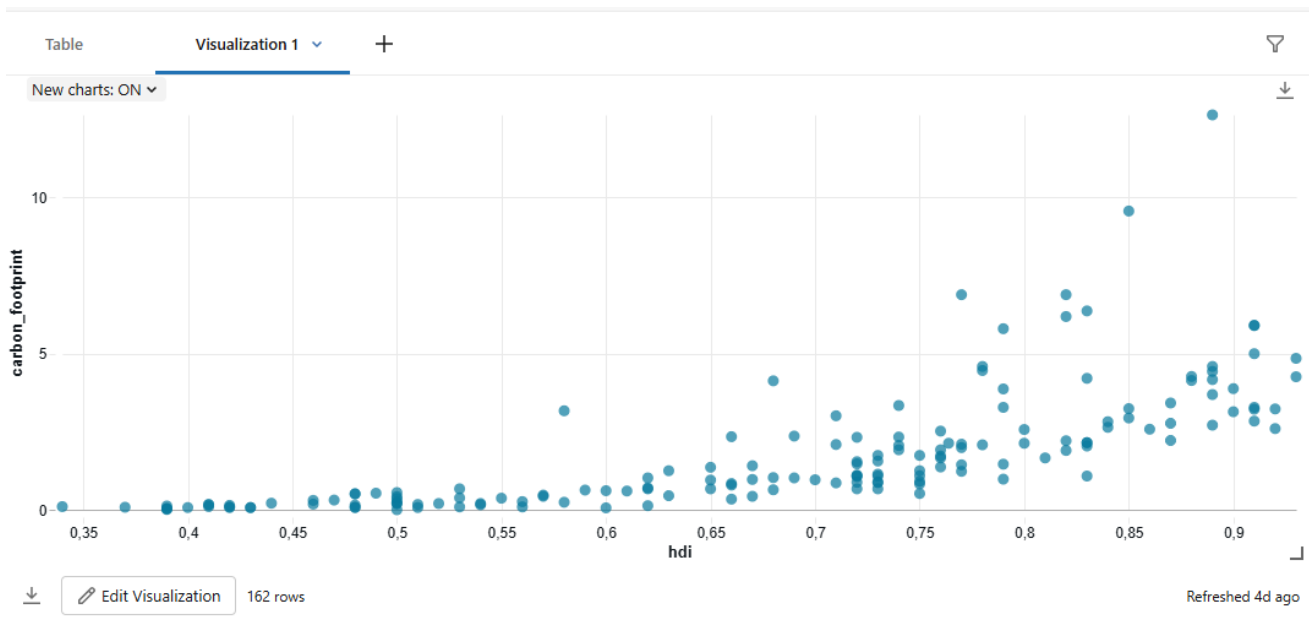


Figura 17 — Gráfico de Dispersão a partir da query `SELECT hdi, carbon_footprint, country, region FROM countries_environmental_analysis ORDER BY hdi`. Eixo X: hdi; eixo Y: carbon_footprint.

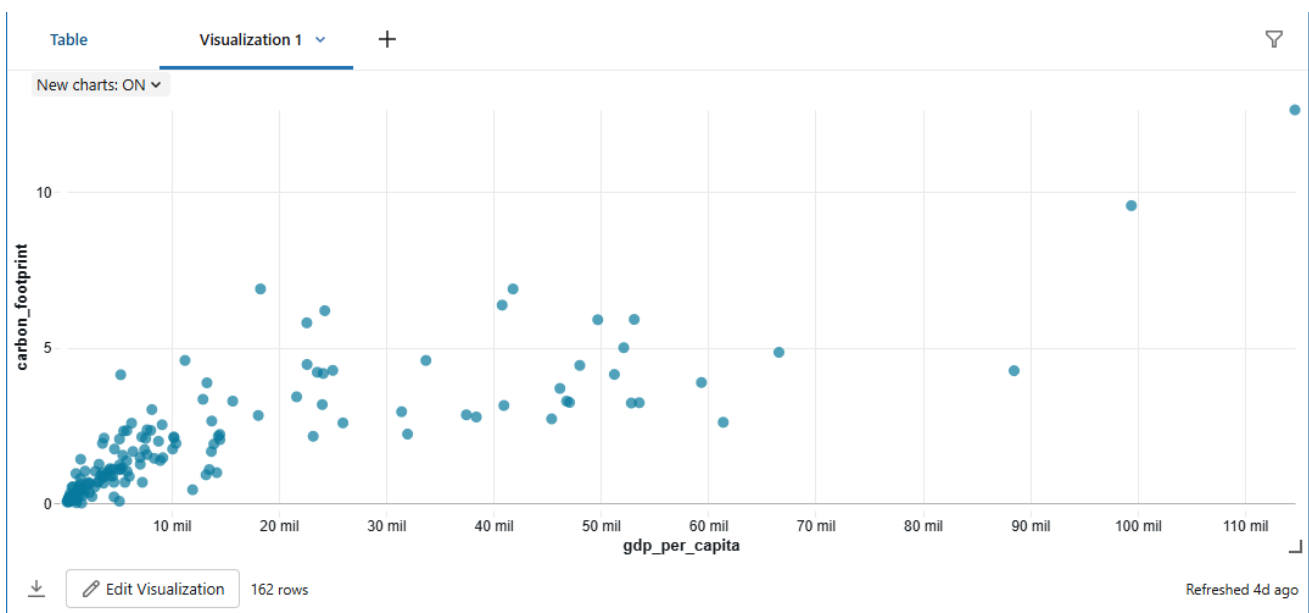


Figura 18 — Gráfico de Dispersão a partir da query `SELECT gdp_per_capita, carbon_footprint, country, region FROM countries_environmental_analysis ORDER BY gdp_per_capita`. Eixo X: gdp_per_capita; eixo Y: carbon_footprint.

10. Interpretação dos resultados

Os resultados respondem de forma consistente à pergunta central do trabalho: IDH e PIB Per Capita apresentam associação com componentes da pegada ecológica, porém a intensidade dessa relação varia substancialmente entre os componentes.

A pegada de carbono é o componente que apresenta a relação mais forte com desenvolvimento. A correlação de 0,824 entre PIB Per Capita e Pegada de Carbono indica associação linear positiva elevada na amostra. A correlação de 0,699 entre IDH e Pegada de Carbono aponta comportamento semelhante. Essa leitura também é sustentada pelas agregações: a média de Pegada de Carbono cresce de 0,202 para 3,869 entre as faixas de IDH baixo e muito alto e de 0,285 para 4,573 entre as faixas de PIB abaixo de 2 mil e acima de 30 mil.

Área de Cultivo apresenta uma relação intermediária com os indicadores de desenvolvimento. As correlações de 0,567 com IDH e 0,506 com PIB Per Capita sugerem uma associação positiva moderada. As médias por faixa também crescem de forma progressiva.

Pegada de Peixe e Área de Pastagem apresentam correlações baixas. Isso indica que, nesta base, IDH e PIB Per Capita isoladamente explicam pouco da variação linear desses componentes. A análise dos *rankings* reforça a necessidade de considerar características geográficas e produtivas específicas de cada país.

Os resultados regionais devem ser interpretados com cautela porque os grupos possuem tamanhos diferentes. A América do Norte, por exemplo, possui somente dois países na Gold, enquanto a África possui 47. Portanto, as médias regionais são úteis para descrição, mas não devem ser entendidas como estimativas com a mesma estabilidade estatística.

11. Conclusões

O MVP atingiu o objetivo de construir um pipeline de dados ponta a ponta em ambiente de nuvem, com ingestão, persistência, tratamento, modelagem, qualidade e análise. A arquitetura Bronze/Silver/Gold tornou explícita a evolução do dado e permitiu separar o dado recebido, o dado tratado e os produtos analíticos.

Do ponto de vista do problema, os resultados indicam que o desenvolvimento socioeconômico está mais fortemente associado à pegada de carbono do que às pegadas de pesca e pastagem. A Área de Cultivo ocupa uma posição intermediária. Portanto, não é adequado tratar a pegada ecológica como uma dimensão ambiental homogênea: diferentes componentes apresentam relações distintas com IDH e PIB Per Capita.

Do ponto de vista de Engenharia de Dados, a persistência das tabelas auxiliares Gold permite que os resultados sejam reutilizados por consultas SQL, visualizações e futuros dashboards sem repetir as etapas de limpeza e transformação.

12. Limitações e pontos de melhoria

A principal limitação é o caráter transversal do conjunto de dados. Como a base representa um recorte por país e não uma série histórica, as relações encontradas não permitem avaliar evolução temporal nem estabelecer causalidade.

A presença de valores ausentes reduziu a base analítica de 188 para 162 países. Optou-se por não realizar imputação das variáveis centrais, evitando criar valores artificiais em indicadores nacionais.

Outra limitação é que IDH e PIB Per Capita não representam todos os fatores capazes de influenciar as pegadas. Matriz energética, estrutura industrial, urbanização, clima, território, comércio internacional e padrões de consumo podem contribuir para explicar os resultados.

Como evolução do projeto, recomenda-se incorporar séries históricas e novas variáveis socioeconômicas e energéticas, automatizar a execução do pipeline com Workflows, criar controles formais de qualidade e disponibilizar dashboards alimentados diretamente pelas tabelas Gold.

A documentação original e a licença do *dataset* devem ser explicitadas na versão final, sempre que utilizada, caso estejam disponíveis na página de origem. O notebook comprova o repositório GitHub utilizado para ingestão, mas não contém informação da licença original do conjunto de dados.

13. Autoavaliação

O desenvolvimento deste MVP consolidou conhecimentos de Engenharia de Dados aplicados a um fluxo completo em nuvem. A construção das camadas Bronze, Silver e Gold permitiu trabalhar não apenas com análise, mas também com persistência, rastreabilidade, qualidade, modelagem e organização do consumo analítico.

Um dos pontos mais relevantes do projeto foi a evolução da solução para que a maior parte das análises fosse realizada sobre tabelas Delta efetivamente armazenadas no Databricks. PySpark permaneceu

essencial na preparação e materialização dos dados, enquanto Spark SQL passou a ser utilizado como principal mecanismo de consulta dos produtos analíticos.

A adaptação necessária na etapa de ingestão também foi importante para o aprendizado. A leitura do arquivo por Pandas e posterior conversão para Spark permitiu adequar o notebook ao ambiente real de execução sem alterar a arquitetura proposta.

Considero que os objetivos definidos foram atingidos: o pipeline foi executado de ponta a ponta, as tabelas foram persistidas, os problemas de qualidade foram identificados e tratados de maneira rastreável, e as perguntas de análise puderam ser respondidas a partir das tabelas Gold. Como continuidade, o projeto pode ser aprimorado com automação, dados temporais e novos indicadores explicativos.

Este projeto foi desenvolvido originalmente em Python, e sua reestruturação para uma abordagem baseada majoritariamente em SQL representou um dos principais desafios encontrados durante o desenvolvimento. Nesse processo, utilizei ferramentas de Inteligência Artificial como apoio para esclarecer dúvidas e auxiliar na reorganização da solução. Essa experiência contribuiu para ampliar minha compreensão sobre o trabalho com DaaS (*Data as a Service*), além de proporcionar maior familiaridade com a plataforma Databricks e com a utilização integrada de Python, PySpark e Spark SQL em um pipeline de dados